

From: homeworkbryn@gmail.com
Subject: Web Dev, Python, Bash & Grep - Beginner Guide (Fixed)
Date: Mar 28, 2026 at 5:59:38 AM
To: bryn.kai-hendricks@outlook.com

Web Dev, Python, Bash & Grep

A Complete Beginner's Guide for Dummies

HTML (HyperText Markup Language) is the backbone of every website. Think of it like the skeleton of a body — it gives structure to web pages. Every website you've ever visited is built on HTML.

When you visit a website, your browser (Chrome, Firefox, Safari) downloads an HTML file and reads it like instructions. The HTML tells the browser: "Put a heading here, a paragraph there, an image over here." It doesn't make things pretty or interactive — it just defines what's on the page.

— HTML uses "tags" to define content. Tags are words wrapped in angle brackets. Most tags come in pairs: an opening tag and a closing tag (with a `<` / `>`).

```
<h1>This is a big heading</h1>
```

```
<p>This is a paragraph of text.</p>
```

```

```

-
-
-
-
-
-
-

— A tag plus its content equals an "element." So `<p>Hello</p>` is a paragraph element. — Extra information you attach to a tag. They go inside the opening tag.

```
  
<a href="https://google.com" target="_blank">Open Google</a>
```

-
-
-
-

— Every HTML page follows this basic structure:

```
<!DOCTYPE html>  
  
<html>  
  
  <head>  
  
    <title>My Page Title</title>  
  
  </head>  
  
  <body>  
  
    <h1>Welcome!</h1>  
  
    <p>This is my first web page.</p>  
  
  </body>  
  
</html>
```

-
-
-
-

— Tags go inside other tags, like folders inside folders. The inner tag is a "child" of the outer tag.

```
<div>

  <h2>My Section</h2>

  <p>Some text inside the section.</p>

</div>
```

The `<h2>` and `<p>` are children of the `<div>`. This parent-child relationship is how you organize a page.

A `<div>` is like an invisible box you use to group content together. Think of it like a shipping container — you can put anything inside it and move it around as one unit.

-
-
-
-

```
<div class="product-card">

  

  <h2>Running Shoe</h2>

  <p class="price">$89.99</p>

  <button>Add to Cart</button>

</div>
```

Think: `div` = "division" of the page. Each div is a section you can style and control independently.

— When you write `class="product-card"` , you're giving that div a name. CSS and JavaScript use this name to find and style/control it. One class can be used on many elements. — Like `class` , but unique. Only one element on the page should have a given `id` . Use it when you need to target one specific thing.

```
<div id="main-header">This is the one and only header</div>
```

```
<div class="card">Card 1</div>
```

```
<div class="card">Card 2</div>
```

```
<div class="card">Card 3</div>
```

CSS (Cascading Style Sheets) controls how HTML looks — colors, sizes, fonts, spacing, layouts. If HTML is the skeleton, CSS is the skin and clothes.

You write "rules" that say "find this element and make it look like this."

```
.product-card {  
  background-color: white;  
  
  border-radius: 8px;  
  
  padding: 20px;  
  
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
  
  width: 300px;  
  
}
```

This says: "Find every element with the class `product-card` and give it a white background, rounded corners, inner spacing, a subtle shadow, and make it 300 pixels wide."

```
h1 { }                /* All <h1> tags */

.card { }              /* All elements with class="card" */

#main-header { }      /* The one element with id="main-header" */

div p { }              /* All <p> tags inside a <div> */

.card:hover { }        /* A .card when the mouse hovers over it */


color: #333333;         /* Text color (dark gray) */
background-color: #f0f0f0; /* Background color (light gray) */

background-color: rgb(255, 0, 0); /* Red using RGB */

background-color: rgba(0, 0, 0, 0.5); /* Black at 50% opacity */


width: 300px;           /* Fixed width in pixels */
width: 50%;             /* Half the parent's width */

height: 200px;          /* Fixed height */

max-width: 1200px;      /* Won't grow beyond this */

min-height: 100vh;      /* At least the full screen height */


padding: 20px;          /* Space INSIDE the box (between border and content) */
margin: 10px;           /* Space OUTSIDE the box (between this box and neighbors) */

padding: 10px 20px;     /* 10px top/bottom, 20px left/right */

margin: 0 auto;         /* Center the element horizontally */
```

Think of it like a picture frame: is the matting inside the frame, is the wall space around the frame.

```
font-size: 16px;           /* Text size */
font-weight: bold;         /* Bold text (or 700) */

font-family: Arial, sans-serif; /* Font choice */

text-align: center;        /* Center the text */

line-height: 1.5;          /* Space between lines */


border: 1px solid #ccc;     /* 1px gray border */
border-radius: 8px;         /* Rounded corners */

border-bottom: 2px solid blue; /* Only bottom border */


.container {
  display: flex;             /* Activate flexbox */

  justify-content: center; /* Center items horizontally */

  align-items: center;      /* Center items vertically */

  gap: 20px;                /* Space between items */

  flex-direction: row;      /* Items in a row (default) */

  flex-wrap: wrap;          /* Wrap to next line if no room */
}
```

Flexbox is the most important layout tool in modern CSS. Think of it like saying "put these items in a row (or column) and here's how to space them out."

— Making your page look good on any screen size:

```
@media (max-width: 768px) {  
  .container {  
  
    flex-direction: column; /* Stack vertically on small screens */  
  
  }  
  
  .card {  
  
    width: 100%; /* Full width on mobile */  
  
  }  
  
}
```

JavaScript (JS) makes web pages interactive. It's the brain that responds to clicks, updates content, fetches data, and controls behavior. Every modern website uses it.

JavaScript runs in the browser. When you visit a website, the browser downloads the HTML, CSS, and JavaScript files. The JS code then runs on your computer, making the page interactive without needing to reload.

— Containers that store data.

```
let name = "Bryn"; /* Can be changed later  
const age = 20; /* Cannot be changed (constant)  
  
let score = 0; /* A number
```

```
let isLoggedIn = true;      // A boolean (true or false)
```

-
-
-

— Reusable blocks of code. Like a recipe you can follow over and over.

```
function greet(name) {  
  return "Hello, " + name + "!";  
}
```

```
greet("Bryn");    // Returns "Hello, Bryn!"
```

```
greet("Alex");    // Returns "Hello, Alex!"
```

```
// Arrow function (shorter syntax, same thing)
```

```
const add = (a, b) => a + b;
```

```
add(5, 3);        // Returns 8
```

— Making decisions.

```
let temperature = 30;
```

```
if (temperature > 25) {
```

```
  console.log("It's hot outside!");
```

```
} else if (temperature > 15) {
```

```
  console.log("It's nice outside.");
```

```
} else {  
  
    console.log("It's cold outside!");  
  
}
```

— Ordered lists of things.

```
let fruits = ["apple", "banana", "cherry"];  
fruits[0];           // "apple" (first item, counting starts at 0)  
  
fruits.length;       // 3  
  
fruits.push("date");  // Add to the end  
  
fruits.includes("banana"); // true  
  
// Loop through every item  
  
fruits.forEach(fruit => {  
  
    console.log(fruit);  
  
});  
  
// Transform every item  
  
let upperFruits = fruits.map(f => f.toUpperCase());  
  
// ["APPLE", "BANANA", "CHERRY", "DATE"]  
  
// Filter items  
  
let longNames = fruits.filter(f => f.length > 5);
```

```
// ["banana", "cherry"]
```

— Collections of key-value pairs (like a dictionary).

```
let student = {  
  name: "Bryn",  
  
  age: 20,  
  
  major: "Computer Science",  
  
  grades: [85, 92, 78]  
  
};  
  
student.name;           // "Bryn"  
  
student.grades[0];       // 85  
  
student.major = "Math"; // Change a value
```

— Reacting to user actions (clicks, typing, scrolling).

```
const button = document.getElementById("myBtn");  
button.addEventListener("click", () => {  
  
  document.getElementById("output").textContent = "You clicked!";  
  
});
```

— JavaScript can change the HTML page on the fly.

```
// Find an element  
const heading = document.querySelector("h1");
```

```
// Change its text

heading.textContent = "New Title!";

// Change its style

heading.style.color = "blue";

heading.style.fontSize = "48px";

// Add a new element

const newParagraph = document.createElement("p");

newParagraph.textContent = "I was added by JavaScript!";

document.body.appendChild(newParagraph);

// Remove an element

heading.remove();

// Get data from an API

const response = await fetch("https://api.example.com/tasks");

const tasks = await response.json();

console.log(tasks);

// Send data to a server

await fetch("/api/tasks", {
```

```
method: "POST",

headers: { "Content-Type": "application/json" },

body: JSON.stringify({ title: "Do homework", dueDate: "2026-04-01" })

});
```

HTML = the house structure, CSS = the paint and decor, JavaScript = the electricity and plumbing that makes things work.

Python is a general-purpose programming language known for being easy to read and learn. It's used for web backends, data science, AI, automation, and more. Many universities teach it as a first language because its syntax reads almost like English.

JavaScript runs in the browser (frontend). Python typically runs on a server or your computer (backend). You can build websites with both, but they're used for different parts.

— Python is simpler than JavaScript. No `let` or `const` needed.

```
name = "Bryn"

age = 20

pi = 3.14159

is_student = True    # Note: True/False are capitalized in Python
```

— Python uses spaces (usually 4) instead of `{ }` braces to define code blocks. This is the biggest gotcha for beginners.

```
# Correct (4 spaces of indentation)

if age >= 18:
```

```
print("You're an adult")
```

```
print("You can vote")
```

```
if age >= 18:
```

```
    print("You're an adult")    # Not indented = broken
```

— Like JavaScript arrays.

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits[0]          # "apple"
```

```
len(fruits)        # 3
```

```
fruits.append("date") # Add to end
```

```
for fruit in fruits:
```

```
    print(fruit)
```

```
upper_fruits = [f.upper() for f in fruits]
```

```
long_names = [f for f in fruits if len(f) > 5]
```

— Like JavaScript objects (key-value pairs).

```
student = {  
    "name": "Bryn",  
  
    "age": 20,  
  
    "major": "Computer Science"  
}  
  
student["name"]          # "Bryn"  
  
student["gpa"] = 3.8      # Add new key  
  
def calculate_average(grades):  
    total = sum(grades)  
  
    return total / len(grades)  
  
my_grades = [85, 92, 78, 95, 88]  
  
avg = calculate_average(my_grades)  
  
print(f"Your average is {avg}")
```

— Python's way of putting variables inside text (the `f` before the quotes):

```
name = "Bryn"
age = 20

print(f"My name is {name} and I'm {age} years old.")
```

```
score = 85

if score >= 90:

    grade = "A"

elif score >= 80:

    grade = "B"

elif score >= 70:

    grade = "C"

else:

    grade = "F"

print(f"Your grade is {grade}") # "Your grade is B"
```

— Python has thousands of add-on packages:

```
import math

print(math.sqrt(144))    # 12.0


import random

print(random.randint(1, 100)) # Random number between 1 and 100


from datetime import datetime

now = datetime.now()

print(now.strftime("%B %d, %Y")) # "March 28, 2026"


# Write to a file
with open("notes.txt", "w") as f:

    f.write("Hello, world!\n")

    f.write("Second line.\n")




---



with open("notes.txt", "r") as f:

    content = f.read()

    print(content)
```

Bash is the language of the terminal (also called the command line, shell, or console). It's how you talk directly to your computer without a graphical interface. Every Mac, Linux computer, and server uses it. On Windows, you can use Git Bash or WSL.

-
-
-
-

```
pwd                                # Print Working Directory - shows where you are
                                   # Example output: /home/bryn/Documents

ls                                 # List - shows files and folders in current directory

ls -l                             # Long format - shows sizes, dates, permissions

ls -la                            # Also show hidden files (ones starting with .)

ls -lh                            # Human-readable file sizes (KB, MB, GB instead of bytes)

cd Documents                      # Change Directory - move into the Documents folder

cd ..                             # Go up one level (back to parent folder)

cd ~                              # Go to your home directory

cd /                              # Go to the root of the entire filesystem
```

Think of `cd` like double-clicking a folder, and `ls` like looking at what's inside.

```
mkdir my-project                 # Make a new directory (folder)

mkdir -p a/b/c                  # Make nested folders all at once
```

```
touch notes.txt          # Create an empty file (or update its timestamp)

cp file.txt backup.txt   # Copy a file

cp -r my-folder backup-folder # Copy a folder and everything in it

mv old-name.txt new-name.txt # Rename a file

mv file.txt ~/Desktop/    # Move a file to the Desktop

rm file.txt               # Delete a file (PERMANENT - no recycle bin!)

rm -r folder/             # Delete a folder and everything in it

rm -rf folder/            # Force delete - no confirmation prompts (DANGEROUS)
```

rm is permanent. There is no undo. Always double-check before deleting.

```
cat file.txt             # Print the entire file to the screen

head file.txt            # Show the first 10 lines

head -n 20 file.txt      # Show the first 20 lines

tail file.txt            # Show the last 10 lines

tail -f log.txt          # Follow a file live (great for watching logs)

less file.txt            # View a file page by page (press q to quit)

wc -l file.txt           # Count the number of lines in a file
```

The pipe **|** takes the output of one command and feeds it into another. This is one of Bash's superpowers.

```
ls -la | head -5          # List files, but only show the first 5 lines
cat file.txt | wc -l      # Count lines in a file

history | tail -20        # Show your last 20 commands

echo "Hello" > file.txt   # Write "Hello" to file (overwrites!)

echo "World" >> file.txt   # Append "World" to file (adds to end)
```

Think of | like a conveyor belt: the output of the left side slides into the input of the right side.

```
clear                    # Clear the terminal screen
history                  # Show all your past commands

whoami                   # Show your username

date                     # Show current date and time

df -h                   # Show disk space usage

du -sh folder/          # Show total size of a folder

top                      # Live view of running processes (q to quit)

kill 12345               # Stop a process by its ID number

chmod 755 script.sh     # Change file permissions (make it executable)


node app.js              # Run a JavaScript file with Node.js
python3 script.py        # Run a Python script

npm install              # Install JavaScript packages
```

```
npm run dev          # Run a development server
```

```
pip install requests # Install a Python package
```

```
echo $HOME           # Print the value of the HOME variable
```

```
export MY_VAR="hello" # Set a variable for this session
```

```
echo $MY_VAR         # Print it
```

```
DATABASE_URL=postgresql://user:pass@localhost/mydb
```

```
mkdir project && cd project          # Do both - only cd if mkdir succeeds
```

```
npm install && npm run build         # Install, then build
```

```
command1 ; command2                 # Run both regardless of success
```

```
npm run dev &                       # Run in background (& at the end)
```

Grep stands for global regular expression print. It searches through files for lines that match a pattern. Think of it as Ctrl+F for your entire computer.

-
-
-
-

```
grep "error" log.txt
```

This searches `log.txt` and prints every line containing the word "error."

```
grep "error" log.txt          # Find lines with "error"
grep -i "error" log.txt       # Case-insensitive (finds Error, ERROR, error)

grep -n "error" log.txt       # Show line numbers

grep -c "error" log.txt       # Count matching lines

grep -r "TODO" .              # Search recursively in all files under current
folder

grep -l "error" *.txt         # Only show filenames that match (not the lines)

grep -v "debug" log.txt       # Invert - show lines that DON'T match

grep -w "cat" file.txt        # Whole word only (won't match "catalog")


grep "function" *.js          # Search all JavaScript files in current folder
grep -r "import" src/         # Search all files in src/ folder and subfolders

grep -rn "TODO" --include="*.ts" .  # Search only TypeScript files, show line
numbers
```

This is where grep really shines - filtering the output of other commands:

```
# Find running Node processes  
ps aux | grep "node"
```

```
npm list | grep "express"
```

```
history | grep "git"
```

```
ls -la | grep -i ".pdf"
```

```
cat server.log | grep -c "ERROR"
```

```
grep -rn "calculateGrade" --include="*.ts" .
```

Grep supports patterns (called "regular expressions" or "regex") for flexible searching:

```
grep "^Error" log.txt          # Lines that START with "Error"
grep "done$" log.txt          # Lines that END with "done"

grep "error\|warning" log.txt  # Lines with "error" OR "warning"

grep "task-[0-9]" log.txt      # "task-" followed by any digit (task-1, task-9)

grep "2026-03-.." log.txt      # Any date in March 2026

grep -E "[0-9]{3}-[0-9]{4}" contacts.txt # Phone number pattern (555-1234)
```

```
# Find all TODO comments in your codebase
```

```
grep -rn "TODO\
```

```
    FIXME\
```

```
HACK" --include="*.ts" --include="*.tsx" .
```

```
grep -rn "function calculateGrade" --include="*.ts" .
```

```
grep "ERROR" /var/log/app.log
```

```
grep -rn "\.product-card" --include="*.css" .
```

```
grep -rn "app\\.\\(get\\  
  
    post\\                                put\\  
  
delete\\)" --include="*.ts" server/
```

```
grep -rl "export default function" --include="*.tsx" src/ | wc -l
```

Technology	What It Is	Analogy
	Structure (what's on the page)	The skeleton/frame of a house
	Style (how it looks)	The paint, furniture, and decor
	Behavior (what it does)	The electricity and plumbing
	A box to group HTML elements	A shipping container
	Backend logic, scripts, data, AI	A Swiss Army knife

**Command-line
interface to
your computer**

**Talking
directly to
your
computer**

**Search engine
for file contents**

**Ctrl+F for
your whole
computer**

When you visit a website:

1. defines the content - headings, paragraphs, buttons, images
2. makes it look good - colors, spacing, responsive layout
3. makes it interactive - handles clicks, loads data, updates the page
4. (or another backend language) runs on the server - handles databases, authentication, business logic
5. is how developers run, deploy, and manage everything on the server
6. is how developers find things in codebases and debug problems



**WebDev_Python_Bash_Gr
ep_Guide.md**
23 KB